Property Management System: Project Breakdown & Roles

###1. Problem Domain: UML Requirements

We are modeling a **Property Management System**. This high-level view includes the attributes and relationships required for our UMLET diagram

Entity	Primary Attributes	Relationship
Property	propertyId, address, description, rentalAmount, isAvailable	Managed by Manager; occupied by Tenant.
Tenant	tenantId, firstName, lastName, email, phoneNumber	Rents a Property; signs a Lease.
Manager	managerId, name, contactNumber, employeeId	Oversees Properties and Leases.
Lease	leaseId, startDate, endDate, monthlyRent, deposit	Connects one Tenant to one Property.
Payment	paymentId, amount, paymentDate, paymentMethod	Linked to a specific Lease or Tenant.
Maintenance	requestId, description, status (Open/Closed), cost	Requested for a specific Property.

###2. Mandatory Coding Rules (Read Carefully)

To ensure we don't lose marks, everyone must follow these technical requirements:

- **The 'Code to Abstraction' Rule:** When implementing repositories, always code to the interface (IRepository), not the concrete implementation.
- **Builder Pattern:** All Entities must be implemented using the **Builder Pattern**.
- **TDD Assessment:** Your Pull Requests **must** include the Factory Test and Repository Test classes. This is how we are assessed on Test-Driven Development.
- **File Headers:** Every Java source file must include this header:

  Java

```
/* * [FileName].java
 * [Class Description]
 * Author: [Your Name] ([Student Number])
 * Date: [Current Date]
 */
```

---

3. Role Selection & Assignments

Each member is responsible for the full implementation (Entity, Factory, Repository) of their assigned domain.

1. **Team Lead: Property** Entity. Also responsible for the Core IRepository<T, ID> setup and GitHub Management.
2. **Member 2: Tenant** Management.
3. **Member 3: Payment** / Financial Records.
4. **Member 4: Lease** / Legal & Contracts.
5. **Member 5: Maintenance** & Operations.
6. **Member 6: Manager** / Personnel.

---

4. Project Timeline

Milestone	Due Date	Task Description
Setup & Branching	**ASAP**	Create a branch named after your **student number**.
Entity & Factory	**March 20 (Friday)**	Complete your Entity (Builder Pattern) and Factory (TDD).
Repository Milestone	**March 22 (Sunday)**	Implement your Repository Interface and Implementation Class.
Final Submission	**March 27 (Friday)**	All code merged to master and submitted to Blackboard by 23:59.

---

5. GitHub Workflow

1. **Work on your branch:** Do not push directly to master.
2. **Push & Pull Request:** Once your task is error-free, push to your upstream and send a **Pull Request** to the Team Lead.
3. **Sync:** After the Lead merges a milestone, pull the updated master to your local machine to stay in sync.

Here is a baseline template you guys can use, please note if it goes against how the lecturer taught us we may lose marks.

1. Tenant.java (Domain Package)

This implementation uses the **Builder Pattern** as required by Section D1.

Java
```
package domain;

/*
 * Tenant.java
 * Entity for the Tenant
 * Author: [Member Name] ([Student Number])
 * Date: 19 March 2026
 */
public class Tenant {
    private String tenantId;
    private String firstName;
    private String lastName;
    private String email;
    private String phoneNumber;

    private Tenant() {}

    private Tenant(Builder builder) {
        this.tenantId = builder.tenantId;
        this.firstName = builder.firstName;
        this.lastName = builder.lastName;
        this.email = builder.email;
        this.phoneNumber = builder.phoneNumber;
    }

    public String getTenantId() { return tenantId; }
    public String getFirstName() { return firstName; }
    public String getLastName() { return lastName; }
    public String getEmail() { return email; }
    public String getPhoneNumber() { return phoneNumber; }

    public static class Builder {
        private String tenantId;
        private String firstName;
        private String lastName;
        private String email;
        private String phoneNumber;
```

```
        public Builder setTenantId(String tenantId) {
            this.tenantId = tenantId;
            return this;
        }

        public Builder setFirstName(String firstName) {
            this.firstName = firstName;
            return this;
        }

        public Builder setLastName(String lastName) {
            this.lastName = lastName;
            return this;
        }

        public Builder setEmail(String email) {
            this.email = email;
            return this;
        }

        public Builder setPhoneNumber(String phoneNumber) {
            this.phoneNumber = phoneNumber;
            return this;
        }

        public Builder copy(Tenant tenant) {
            this.tenantId = tenant.tenantId;
            this.firstName = tenant.firstName;
            this.lastName = tenant.lastName;
            this.email = tenant.email;
            this.phoneNumber = tenant.phoneNumber;
            return this;
        }

        public Tenant build() {
            return new Tenant(this);
        }
    }

    @Override
    public String toString() {
        return "Tenant{" + "tenantId='" + tenantId + '\'' + ", firstName='"
+ firstName + '\'' + '}';
    }
}
```

---

2. TenantFactory.java (Factory Package)

This class is used to create instances of the Tenant entity.

Java

```
package factory;

import domain.Tenant;
import util.Helper; // Assuming a helper for ID generation

/*
 * TenantFactory.java
```

```
 * Factory for Tenant Entity
 * Author: [Member Name] ([Student Number])
 * Date: 19 March 2026
 */
public class TenantFactory {
    public static Tenant createTenant(String firstName, String lastName,
String email, String phoneNumber) {
        if (Helper.isNullOrEmpty(firstName) ||
Helper.isNullOrEmpty(lastName)) {
            return null;
        }

        String tenantId = Helper.generateId();

        return new Tenant.Builder()
                .setTenantId(tenantId)
                .setFirstName(firstName)
                .setLastName(lastName)
                .setEmail(email)
                .setPhoneNumber(phoneNumber)
                .build();
    }
}
```

---

3. TenantFactoryTest.java (Test Package)

This is the **TDD component**. Every member must provide this to show the factory works.

Java

```
package factory;

import domain.Tenant;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

/*
 * TenantFactoryTest.java
 * TDD Factory Test
 * Author: [Member Name] ([Student Number])
 * Date: 19 March 2026
 */
class TenantFactoryTest {
    @Test
    public void testCreateTenantSuccess() {
        Tenant tenant = TenantFactory.createTenant("John", "Doe",
"john@test.com", "0123456789");
        assertNotNull(tenant);
        System.out.println(tenant.toString());
    }

    @Test
    public void testCreateTenantFailure() {
        Tenant tenant = TenantFactory.createTenant("", "Doe",
"john@test.com", "0123456789");
        assertNull(tenant);
    }
}
```